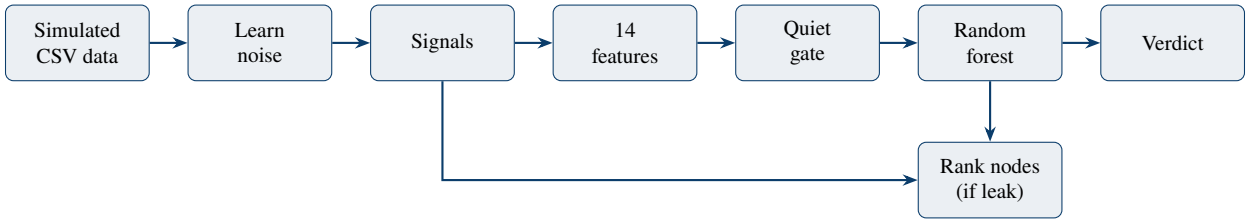
LeakLens

Water Leak Localization System

The ML Pipeline, Explained

The pipeline is mostly **physics-based processing in front of a small machine-learning model**. Only the "which cause?" step is learned from data. The noise levels, the leak ranking and the quiet check are calculations, not trained models. All results are from simulation, not real sensors.



Code lives in `ml_pipeline/`: `dataset.py`, `features.py`, `detector.py`, `classifier.py`, `localizer.py`, `pipeline.py`, `train.py`, `evaluate.py`.

1 Training and testing: the 80/20 split (`split.py`)

If a system is tested on the scenarios it learned from, it looks perfect and tells us nothing. So all 617 scenarios were split once (fixed seed) into **478 for learning (about 80%)** and **139 for testing (about 20%)**.

- **Learned from training scenarios only:** the noise thresholds and the random forest.
- **Measured on test scenarios only:** every result in this document. The system never saw them while learning.
- **Near-copies stay together:** all leaks at the same junction (every size, start time, noise and "hidden behind" variant), and a scenario with its noisy twin, are on the same side. Otherwise the test would be too easy.
- **Stratified:** each network and each kind of event is split about 80/20. The test set has 12 scenarios of every non-leak kind and 67 leaks.
- The dashboard tags every scenario *Unseen test* or *Training*.

2 Data in (`dataset.py`)

- Each scenario has three tables with one row per hour: **node pressure**, **pipe flow** and **metered node demand**. Each network also has a normal-day baseline.
- The network layout is read from the EPANET file. From it we build a **pipe graph** (nodes joined by pipes, with lengths) and an **incidence matrix**, so inflow minus outflow at every node is one matrix multiplication.
- Unreliable readings are dropped: Net3 nodes 61 and 601 and hour 21 swing wildly for a 1% demand change (pump switching).

3 Learn what noise looks like (`features.py: calibrate`)

Each network has 18 *normal runs* that contain only sensor noise; only the training ones (about 14 per network) are used. From them the system learns the noise level of pressure, of each pipe's flow, of each node's demand and of each node's unaccounted flow. The pressure threshold is

$$\tau_P = \max\{0.1 \text{ m},\ 4\,\sigma_P\} \approx 0.5 \text{ m}$$

in all three networks. Every later signal is divided by its noise level, so "1.0" means "right at the noise limit". This lets one set of features work on networks of different size.

4 Build signals (`features.py: node_signals`)

Pressure drop. $r_i(t) = P_i^{\text{normal}}(t) - P_i^{\text{obs}}(t)$. It only counts if it stays high for two consecutive hours: $d_i = \max_t \min\{r_i(t), r_i(t{+}1)\}$. One noisy reading cannot trigger anything.

Unaccounted flow. With pipe flows signed +1 into and −1 out of node i,

$$u_i(t) = \sum_p \text{sign}(p,i)\,\Delta Q_p(t) \;-\; \Delta D_i(t) \;=\; Q_{\ell,i}(t).$$

In noise-free data this is exactly the leak flow at node i and zero elsewhere. A demand change cancels because it is metered.

5 Fourteen features (`features.py: extract`)

One row of 14 numbers per scenario. Flows are divided by total customer demand, so networks of different size give comparable numbers.

Group	Features	Meaning
Pressure drops	`drop_ratio`, `frac_hit`, `median_ratio`, `conc_drop`	largest drop, share of junctions affected, typical drop, is it centred on one node
Pressure rises	`rise_ratio`, `frac_rise`	some nodes gain pressure (as with blockages)
Pipe flows	`flow_frac`	share of pipes with changed flow
Sources	`source_change`, `tank_change`, `pump_drop`	reservoir supply change, tank flow change, pump flow lost (1 = off)
Leak evidence	`unacc_ratio`, `unacc_conc`	largest unaccounted flow, its concentration at one node
Demand evidence	`demand_frac`, `demand_total`	share of nodes with changed metered demand; total change

6 Stage 1: the quiet gate (`quiet_mask`)

If drop_ratio < 1, rise_ratio < 1, unacc_ratio < 1, flow_frac < 0.1 and demand_frac $= 0$, the answer is **normal** and nothing more runs. This removes false alarms from sensor noise. It is also why tiny leaks are reported as normal.

7 Stage 2: the classifier (`classifier.py`, `train.py`)

- **Model:** a scikit-learn **random forest**: 300 decision trees voting, `class_weight="balanced"`, `random_state=0`. "Balanced" matters because leaks are about half the scenarios; it keeps the rarer classes from being ignored.
- **Training data:** 478 rows (the training scenarios), one per scenario. The label is the true event class: normal, leak, demand change, pump/reservoir, tank, blocked pipe, ageing.
- **Interpretable twin:** a depth-4 decision tree is also trained; its readable rules are saved to `data/results/decision_`
- **Output:** a probability for each class. The highest is the verdict; its probability gives the High / Medium / Low confidence label (85% and 60% cut-offs).
- Hyperparameters were *not* tuned.

8 Localization (`localizer.py`)

Only runs when the verdict is "leak". For each junction, with $A_i = u_i^{\text{peak}} / \max_j u_j^{\text{peak}}$ and $D_i = d_i / \max_j d_j$,

$$S_i = A_i + 0.3\, D_i.$$

Nodes are sorted by S_i; the first is the Top-1 prediction. Error is the number of pipes (and metres) between the predicted and the true node. This step is *not trained*. Smoothing the scores over neighbouring nodes was tested and gave no improvement on the test leaks, so it is off.

9 How it is checked (`evaluate.py`)

- **Cause classification:** fitted on the training scenarios, scored on the 139 test scenarios. Also **leave-one-network-out**: train on the training scenarios of two networks, test on the third network's test scenarios.
- **Detection:** compared with the old fixed 0.5 m alarm, on the test scenarios.
- **Localization:** the 60 scored test leaks, compared with the first version, with and without demand meters, and pressure only.
- Training-set numbers are also computed, only to show the gap; time per scenario is about 22 ms.

Result on unseen test scenarios	First version	Final
Leak exact node ranked first (Top-1, 60 leaks)	58.3%	**96.7%**
Real leak in first five nodes (Top-5)	71.7%	**98.3%**
False alarms on normal test days (12 days)	41.7%	**0.0%**
Right cause identified (139 test scenarios)	—	**82.7%**
Right cause on the training scenarios, for comparison	—	95.6%

10 Weak points to know

- The model is **small and cheap**: 14 features, 478 training rows, no tuning.
- The **test set is small**: 139 scenarios, only 60 scored leaks (one miss is about 1.7 points) and 12 normal days. Percentages have wide margins of error, and the split was made once.
- **Settings chosen earlier:** the ranking weight (0.3) and the quiet-gate cut-offs were chosen while looking at all scenarios, before the split existed. They were not re-tuned afterwards.

- **Training versus test gap:** cause accuracy is 95.6% on training and 82.7% on test, a sign the model partly memorises. Pump, tank, blockage and ageing are mixed up with each other most.
- **Unseen network:** leaks are mostly found, but accuracy for causes falls to 49–82%, and without demand meters to about 15% on Net3.
- It **relies on demand meters**. Leaks inside the sensor noise are reported as normal.
- The dashboard's confidence for a *training* scenario is optimistic; use the Unseen test tag.
- All of this is simulation, not real sensor data.